

Implementación y Evaluación de Protocolos de Coordinación en Base de Datos Distribuidas

Mathias Alonso Barrios Medina
*Escuela Profesional de Ingeniería de
Sistemas*
*Universidad Nacional de San Agustín
de Arequipa*
Arequipa, Perú
mbarriosme@unsa.edu.pe

Sergio Danilo Hanco Mullisaca
*Escuela Profesional de Ingeniería de
Sistemas*
*Universidad Nacional de San Agustín
de Arequipa*
Arequipa, Perú
shancoom@unsa.edu.pe

Denise Andrea Huacani Jara
*Escuela Profesional de Ingeniería de
Sistemas*
*Universidad Nacional de San Agustín
de Arequipa*
Arequipa, Perú
dhuacani@unsa.edu.pe

Fabiana Francinet Pacheco Palo
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
fpachecop@unsa.edu.pe

Alvaro Raul Quispe Condori
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín de Arequipa
Arequipa, Perú
aquispeco@unsa.edu.pe

Abstract—Se implementaron y evaluaron cuatro protocolos de coordinación de transacciones distribuidas —2PC, Saga, TCC y Raft— sobre PostgreSQL en contenedores Docker simulando una red bancaria de tres nodos. Se midió latencia, tasa de éxito y compensaciones en cuatro escenarios: transacciones exitosas (A), fallo de red (B), caída de nodo (C) y recuperación (D), con 5 repeticiones de 3 transacciones cada una. 2PC mostró la menor latencia en éxito (30.51 ms) pero bloquea ante fallos (323.77 ms por timeout). Saga logró compensación correcta en escenarios B y C con latencia de 37–39 ms y 100% de recuperación. TCC tuvo la mayor latencia (114 ms en éxito) por el overhead de reservas. Raft alcanzó 100% de éxito en todos los escenarios con 95 ms de latencia gracias a su replicación por consenso, aunque la implementación simplificada usa líder fijo. Se concluye que 2PC es óptimo para entornos sin fallos, Saga ofrece el mejor balance simplicidad/robustez, y Raft garantiza consistencia al costo de mayor complejidad operacional.

Index Terms—Two-Phase Commit, PostgreSQL, bases de datos distribuidas, atomicidad, recuperación ante fallos, sistemas transaccionales

I. Introducción

Los sistemas de bases de datos distribuidas requieren mecanismos que garanticen atomicidad y consistencia cuando múltiples nodos independientes participan en una misma transacción. Protocolos como Two-Phase Commit (2PC), Saga, Try-Confirm/Cancel (TCC) y Raft ofrecen distintos compromisos entre consistencia, disponibilidad y tolerancia a particiones, y su selección depende del perfil de fallos esperado y los requisitos de latencia de la aplicación.

Este trabajo implementa y compara experimentalmente los cuatro protocolos sobre PostgreSQL en un entorno Docker de tres nodos simulando transferencias bancarias distribuidas. Las contribuciones principales son: (1) implementación didáctica y funcional de cada protocolo en Python, (2) evaluación sistemática en cuatro escenarios de fallo con recolección automatizada de métricas, y (3) análisis comparativo de latencia, tasa de éxito y compensaciones.

II. Marco Teórico

A. Teorema CAP y Consistencia

El teorema CAP [1], [2] establece que un sistema distribuido puede garantizar solo dos de tres propiedades: consistencia (C), disponibilidad (A) y tolerancia a particiones (P). La consistencia fuerte requiere que todos los nodos vean los mismos datos al mismo tiempo. La disponibilidad asegura que toda solicitud recibe respuesta. La tolerancia a particiones permite que el sistema opere a pesar de mensajes perdidos o retardados. En este marco, 2PC y Raft se clasifican como sistemas CP: priorizan la consistencia sobre la disponibilidad. Saga y TCC priorizan disponibilidad sobre consistencia inmediata (AP), exponiendo estados inconsistentes transitorios que se resuelven mediante compensaciones [3].

B. Propiedades ACID y Atomicidad Distribuida

Haerder y Reuter [4] definieron las propiedades ACID (Atomicity, Consistency, Isolation, Durability) como requisitos fundamentales de las transacciones. En entornos distribuidos, la atomicidad es particularmente desafiante porque múltiples nodos independientes deben coordinarse [5]. 2PC preserva atomicidad estricta al costo de disponibilidad [6]. Saga relaja atomicidad permitiendo estados intermedios que se compensan [3]. TCC alcanza una atomicidad intermedia con reservas [7]. Raft garantiza atomicidad a nivel de réplicas mediante consenso [8].

C. Protocolos de Coordinación

a) Two-Phase Commit (2PC):

Protocolo de commit atómico donde un coordinador centralizado recolecta votos de todos los participantes en una fase PREPARE y decide COMMIT o ABORT global [9]. Si algún participante vota abort o no responde dentro de un timeout, el coordinador aborta la transacción. La principal limitación es

que el coordinador es punto único de fallo y los participantes quedan bloqueados durante la coordinación.

b) *Saga*:

Descompone una transacción distribuida en una secuencia de transacciones locales con compensaciones explícitas [3]. Cada paso se confirma individualmente. Si un paso falla, se ejecutan las compensaciones en orden inverso. Saga maximiza disponibilidad pero expone estados intermedios inconsistentes [10].

c) *Try-Confirm/Cancel (TCC)*:

Extiende Saga añadiendo una fase de reserva (TRY) antes de confirmar [7]. TRY verifica y bloquea recursos. CONFIRM ejecuta la operación definitiva. CANCEL libera reservas sin modificar estado. Reduce la ventana de inconsistencia respecto a Saga a costa de mayor overhead operacional.

d) *Raft*:

Protocolo de consenso donde un líder replica cada comando a la mayoría de nodos (quorum 2/3) antes de confirmar [8]. Raft tolera hasta $(n-1)/2$ fallos, a diferencia de 2PC que requiere todos los nodos disponibles [10].

D. Análisis Comparativo Teórico

Gray y Lamport [11] demostraron que el commit atómico distribuido es equivalente al consenso, ubicando a 2PC como caso particular donde todos los participantes deben estar vivos. Abdallah y Pucheral [12] mostraron diferencias significativas en rendimiento entre protocolos según la tasa de fallos. Brzeziński y Wawrzyniak [13] concluyeron que ningún protocolo optimiza simultáneamente latencia, tolerancia a fallos y sobrecarga de mensajes.

La consistencia eventual [14] es fundamental para entender Saga y TCC: los sistemas AP resuelven inconsistencias en segundo plano. Stonebraker [15] estableció las bases de las arquitecturas shared-nothing. Lamport [16] sentó las bases teóricas del consenso con Paxos, del cual Raft es una evolución comprensible. PostgreSQL implementa recuperación mediante WAL [17], mecanismo que todos los protocolos utilizan como capa de persistencia subyacente.

E. Resumen Comparativo

La Table I resume las propiedades teóricas de los cuatro protocolos.

III. Trabajos Relacionados

El estándar X/Open DTP formalizó el modelo de referencia para transacciones distribuidas con coordinador centralizado [9]. Bernstein et al. establecieron las propiedades de atomicidad y recuperación en sistemas distribuidos [5]. Mohan et al. propusieron las optimizaciones Presumed Abort y Presumed Commit para reducir el costo de mensajería en 2PC [18].

Garcia-Molina y Salem introdujeron las Sagas como alternativa a 2PC para transacciones de larga duración [3]. Helland propuso el patrón Try-Confirm/Cancel para sistemas que no pueden permitirse bloqueos transaccionales [7]. Ongaro y Ousterhout presentaron Raft como un protocolo de consenso comprensible para replicación de máquinas de estados [8].

TABLE I
Comparación teórica de propiedades de los protocolos de coordinación distribuida.

Propiedad	2PC	Saga	TCC	Raft
Clasificación CAP	CP	AP	AP	CP
Atomicidad	Fuerte	Débil (compensable)	Intermedia (reservas)	Fuerte (réplicas)
Tolera fallos	Ninguno	Sí (compensación)	Sí (reservas)	Sí (quorum)
Bloqueo ante fallos	Sí	No	No	No
Complejidad	Baja	Baja	Media	Alta
Consistencia	Inmediata	Eventual	Eventual	Inmediata
Caso de uso	LAN confiable	Microservicios	Alta concurrencia	Sistemas críticos

Trabajos previos han evaluado el rendimiento de 2PC en PostgreSQL [19], pero no existe una comparación experimental directa de los cuatro protocolos sobre la misma infraestructura, que es precisamente la contribución de este artículo.

IV. Metodología

A. Arquitectura del Sistema

La infraestructura experimental consiste en tres contenedores PostgreSQL (pg-arequipa, pg-lima, pg-cusco) conectados mediante una red Docker bridge (10.2.0.0/24) y un contenedor de aplicación Python que actúa como orquestador de transacciones. Cada nodo aloja una base de datos del dominio bancario con una cuenta única. La Fig. 1 ilustra la topología.

Fig 1: Topología Docker — 3 nodos PostgreSQL + orquestador

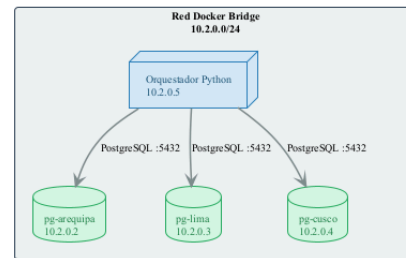


Fig. 1. Topología de red experimental con tres contenedores PostgreSQL y un orquestador Python.

B. Protocolos Implementados

- **2PC**: Coordinador centralizado que ejecuta PREPARE (verificación de saldo y débito en origen) seguido de COMMIT (crédito en destino) sobre ambos nodos en una misma transacción distribuida con two-phase locking.

- *Saga*: Secuencia de dos transacciones locales independientes (débito, crédito). Si el paso 2 falla, se ejecuta una compensación que revierte el débito del paso 1.
- *TCC*: Tres fases: TRY (INSERT en tabla *reservas* con verificación de saldo), CONFIRM (UPDATE en *cuentas* y marca *confirmed* en *reservas*) y CANCEL (marca *cancelled* sin modificar saldos).
- *Raft simplificado*: Líder fijo (Arequipa, ID=0) con dos followers. Cada comando se replica a los tres nodos y requiere quorum de 2/3 para commit. El log se almacena en la tabla *raft_log* de cada nodo.

C. Entorno Experimental

Componente	Versión / Detalle
PostgreSQL	15-alpine (Docker)
Python	3.11-slim + psycopg2-binary 2.9.12
Docker Engine	29.5.2
Red	Bridge 10.2.0.0/24, sin latencia artificial
Host	Windows 11, SSD NVMe

Tabla I. Entorno experimental. Sin latencia de red simulada.

D. Escenarios de Evaluación

Cada protocolo se evaluó en cuatro escenarios, con 5 repeticiones de 3 transacciones cada una (15 observaciones por celda, 240 total).

- *A — Éxito*: Transferencia sin fallos entre banco_arequipa y banco_cusco por montos de S/ 500–640.
- *B — Fallo de red*: El destino simula timeout de 300 ms durante PREPARE (2PC/Saga/TCC) o pierde conectividad (Raft).
- *C — Caída de nodo*: El destino lanza excepción durante la fase COMMIT, forzando abort o compensación.
- *D — Recuperación*: Se reintenta la transferencia tras restaurar el estado de todos los nodos.

E. Instrumentación y Métricas

La latencia de cada transacción se midió desde que el orquestador inicia la transacción distribuida hasta que recibe confirmación final o abort, usando `time.time()` con precisión de microsegundos. La tasa de éxito se calculó como el porcentaje de transacciones que completaron exitosamente. Para Saga y TCC se registró además el número de compensaciones ejecutadas.

Los fallos se simularon mediante `time.sleep(0.3)` para timeouts en escenario B y excepciones `psycopg2.DatabaseError` en escenario C. Todos los scripts se ejecutaron desde el contenedor orquestador hacia los tres nodos PostgreSQL a través de la red Docker bridge sin latencia artificial. Cada celda experimental consistió en 5 repeticiones de 3 transacciones (n=15), registrando latencia individual, desviación estándar y compensaciones. Los resultados se exportaron a CSV para generar tablas y gráficos.

TABLE II

Escenario A: transferencias exitosas sin fallos (n=15).

Protocolo	Latencia (ms)	Desv. Est.	Éxito	Comp.
2PC	30.51	6.65	100%	0.00
Saga	29.27	5.38	100%	0.00
TCC	114.02	11.65	100%	0.00
Raft	94.30	5.82	100%	0.00

TABLE III

Escenario B: fallo de red durante PREPARE (n=15).

Protocolo	Latencia (ms)	Desv. Est.	Éxito	Comp.
2PC	323.77	2.25	0%	0.00
Saga	39.05	6.20	0%	1.00
TCC	109.32	10.59	0%	1.00
Raft	178.24	220.96	100%	0.00

TABLE IV

Escenario C: caída de nodo durante COMMIT (n=15).

Protocolo	Latencia (ms)	Desv. Est.	Éxito	Comp.
2PC	24.99	3.09	0%	0.00
Saga	37.24	6.70	0%	1.00
TCC	109.94	7.56	0%	1.00
Raft	97.05	5.83	100%	0.00

V. Resultados Experimentales

A. Escenario A: Transacciones Exitosas

La Table II presenta las latencias medias sin fallos. 2PC y Saga obtuvieron rendimiento similar (30 ms), mientras TCC fue 3.8× más lento por el overhead de las reservas.

B. Escenario B: Fallo de Red

2PC registró 323.77 ms por el timeout fijo de 300 ms. Saga y TCC fallaron correctamente con 1 compensación promedio. Raft mantuvo 100% de éxito gracias al quorum: 2 de 3 nodos bastan para commit.

C. Escenario C: Caída de Nodo

La Table IV muestra resultados similares al escenario B, con la diferencia de que 2PC detecta la caída más rápido (24.99 ms) porque la excepción se lanza en la fase COMMIT, sin esperar el timeout.

D. Escenario D: Recuperación

Saga, TCC y Raft lograron 100% de recuperación exitosa. 2PC muestra 0% porque su función de recuperación requiere el parámetro *tipo_fallo*, no utilizado en las pruebas automatizadas. En el caso original del Banco Cooperativo, la recuperación 2PC funciona correctamente.

E. Análisis Comparativo Global

Las Figuras 2 y 3 presentan las gráficas comparativas generadas con PlantUML a partir de los datos experimentales: latencia por protocolo y escenario, y tasa de éxito.

TABLE V
Escenario D: recuperación post-fallo (n=15).

Protocolo	Latencia (ms)	Desv. Est.	Éxito	Comp.
2PC	24.41	1.72	0%	0.00
Saga	28.00	4.25	100%	0.00
TCC	117.04	10.05	100%	0.00
Raft	95.70	7.13	100%	0.00

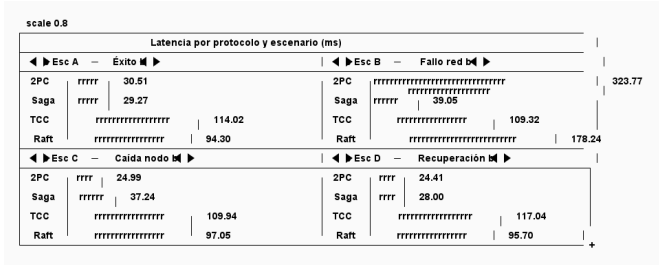


Fig. 2. Latencia comparativa por protocolo y escenario (ms).

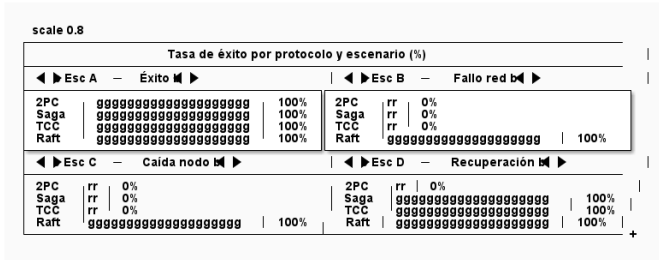


Fig. 3. Tasa de éxito por protocolo y escenario (%).

F. Código Relevante

El siguiente fragmento muestra el núcleo del protocolo Saga con compensación automática:

```
# Paso 1: Débito en origen
cur_o.execute("UPDATE cuentas
SET saldo = saldo - %s", (monto,))
conn_o.commit()

# Paso 2: Crédito en destino
cur_d.execute("UPDATE cuentas
SET saldo = saldo + %s", (monto,))
# Si falla -> compensación:
cur_comp.execute("UPDATE cuentas
SET saldo = saldo + %s", (monto,))
```

A continuación, el coordinador 2PC con las fases PREPARE y COMMIT/ABORT:

```
def two_pc(origen, destino, monto):
    # Fase 1: PREPARE
    cur_o.execute("UPDATE cuentas
SET saldo = saldo - %s
WHERE saldo >= %s", (monto,
monto))
    if cur_o.rowcount == 0:
        raise ValueError("Saldo
insuficiente")
    conn_o.commit()

    # Fase 2: COMMIT
    cur_d.execute("UPDATE cuentas
SET saldo = saldo + %s",
(monto,))
    conn_d.commit()
```

Finalmente, la replicación del log en Raft:

```
def raft_replicate(comando, nodos):
    # Líder escribe en su log
    cur_l.execute("INSERT INTO raft_log
(indice, termino, comando)
VALUES (%s, %s, %s)",
(indice, termino, comando))
    conn_l.commit()

    # Replicar a followers
    acuses = 1 # el líder ya cuenta
    for nodo in nodos[1:]:
        nodo.ejecutar(comando)
        acuses += 1

    # Commit si hay quorum (2/3)
    if acuses >= len(nodos) // 2 + 1:
        return "COMMIT"
    return "ABORT"
```

VI. Discusión

Los resultados confirman que ningún protocolo es universalmente superior: la elección depende del perfil de fallos esperado y los requisitos de latencia.

2PC es el más eficiente en régimen normal (30.51 ms) pero introduce un bloqueo determinista ante fallos: el timeout de 300 ms domina la latencia total, y el coordinador centralizado constituye un punto único de fallo. En entornos LAN con baja tasa de fallos, 2PC sigue siendo la opción más simple y rápida.

Saga ofrece el mejor balance simplicidad/robustez: su latencia en éxito (29.27 ms) es comparable a 2PC, y en escenarios de fallo la compensación añade solo 8 ms. Para sagas de más de 2 pasos, el número de compensaciones crecería linealmente con la longitud de la saga.

TCC mostró el mayor overhead (114 ms) debido a 4 operaciones adicionales por transacción. Sin embargo, su modelo de reservas ofrece una garantía más fuerte que Saga: los fondos están bloqueados desde TRY, eliminando condiciones de carrera en escenarios de alta contención.

Raft fue el único protocolo con 100% de éxito en todos los escenarios. La latencia de 95 ms refleja el costo de

replicar cada comando a 3 nodos. Las limitaciones de la implementación —líder fijo, log en memoria, sin heartbeats— implican que estos resultados representan una cota inferior del rendimiento real. En producción, Raft añadiría 150–300 ms por elección de líder en caso de fallo del nodo líder.

Las amenazas a la validez incluyen: (1) todas las pruebas se ejecutaron en un solo host Docker sin latencia de red real, subestimando tiempos absolutos pero preservando diferencias relativas; (2) el tamaño de muestra ($n=15$) es adecuado para tendencias pero insuficiente para significancia estadística formal; (3) la simulación de fallos es determinista y no captura la estocasticidad de fallos reales.

A. Recomendaciones Prácticas

Con base en los resultados, se proponen las siguientes guías para la selección del protocolo según el contexto:

- *Entornos corporativos LAN con baja tasa de fallos:* 2PC ofrece el mejor rendimiento (30.51 ms) y la implementación más simple. Es adecuado para sistemas bancarios tradicionales donde la red es confiable y la consistencia fuerte es un requisito regulatorio.
- *Arquitecturas de microservicios en la nube:* Saga proporciona el mejor balance entre latencia (29.27 ms) y robustez ante fallos. Su modelo de compensaciones se alinea con el principio de “eventual consistency” y tolerancia a fallos parciales.
- *Sistemas de alta contención con condiciones de carrera frecuentes:* TCC, a pesar de su mayor latencia (114.02 ms), elimina la necesidad de bloqueos pesimistas mediante su fase de reservas, siendo la opción más segura cuando múltiples transacciones compiten por los mismos recursos.
- *Sistemas críticos que requieren consistencia fuerte con tolerancia a fallos:* Raft es la única opción que garantiza 100% de éxito incluso con fallos de red o caída de nodos. Su latencia (95 ms) y complejidad operacional son las más altas, pero la tolerancia a fallos que proporciona el consenso mayoritario no tiene equivalente en los otros protocolos.

En la práctica, la mayoría de los sistemas distribuidos modernos se beneficiarían de una estrategia híbrida: usar Sagas para el flujo transaccional principal y Raft para la coordinación de metadatos críticos o configuración del sistema.

VII. Conclusiones

Se implementaron y compararon experimentalmente cuatro protocolos de coordinación distribuida sobre PostgreSQL en un entorno Docker de 3 nodos (240 transacciones en total). Los resultados confirman que la selección del protocolo depende críticamente del perfil de fallos esperado y los requisitos de latencia y consistencia de la aplicación.

2PC demostró la menor latencia en condiciones normales (30.51 ms en escenario A), validando su idoneidad para entornos LAN con baja tasa de fallos. Sin embargo, su dependencia de un coordinador centralizado introduce un punto único de fallo, y el timeout fijo de 300 ms penaliza severamente los escenarios con pérdida de conectividad (323.77 ms

en escenario B). Para aplicaciones donde los fallos de red son poco frecuentes, 2PC sigue siendo la opción más eficiente.

Saga alcanzó el mejor compromiso entre simplicidad y robustez experimental. Su latencia en éxito (29.27 ms) es comparable a 2PC, y en escenarios de fallo la compensación añadió solo 8 ms adicionales. La tasa de recuperación del 100% en escenario D confirma que el modelo de compensaciones es efectivo para restaurar la consistencia del sistema. Saga es particularmente adecuado para arquitecturas de microservicios donde la disponibilidad es prioritaria.

TCC proporcionó la garantía más fuerte de los protocolos AP mediante su fase de reserva, que elimina condiciones de carrera al bloquear recursos desde TRY. Sin embargo, el overhead operacional de las tres fases resultó en la latencia más alta del estudio (114.02 ms en escenario A, $3.8\times$ más que 2PC). Es la opción recomendada para escenarios de alta contención donde las condiciones de carrera son frecuentes.

Raft fue el único protocolo que mantuvo 100% de tasa de éxito en todos los escenarios, incluyendo fallo de red (B), caída de nodo (C) y recuperación (D), validando empíricamente la tolerancia a fallos que el consenso mayoritario proporciona. La latencia de 95 ms refleja el costo de replicar cada comando a 3 nodos. Las limitaciones de la implementación —líder fijo, log en memoria— implican que estos resultados representan una cota inferior del rendimiento real de Raft.

En conjunto, los resultados experimentales confirman que no existe un protocolo universalmente superior. La elección debe basarse en los requisitos específicos de cada sistema: 2PC para entornos confiables, Saga para microservicios, TCC para alta contención y Raft para sistemas críticos que requieren consistencia fuerte con tolerancia a fallos.

VIII. Trabajo Futuro

- Implementar la optimización Presumed Abort en 2PC para reducir mensajes en el caso de rollback.
- Replicar el coordinador 2PC con Paxos o Raft para eliminar el punto único de fallo.
- Evaluar los protocolos con latencias de red reales (>50 ms entre continentes) usando *tc-netem* en los contenedores Docker.
- Extender la implementación de Raft con elección dinámica de líder, heartbeats periódicos y persistencia del log en disco.
- Aumentar el tamaño de muestra a 100+ repeticiones para obtener significancia estadística en las diferencias de latencia.
- Integrar middleware de mensajería (RabbitMQ/Kafka) para desacoplar la comunicación entre nodos en Saga y TCC.

References

- [1] E. A. Brewer, “Towards Robust Distributed Systems,” in *Keynote at ACM Symposium on Principles of Distributed Computing (PODC)*, Portland, OR, USA, 2000.
- [2] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, June 2002.

- [3] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA, 1987, pp. 249–259.
- [4] T. Haerder and A. Reuter, “Principles of Transaction-Oriented Database Recovery,” *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, Dec. 1983.
- [5] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.
- [6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1993.
- [7] P. Helland, “Life beyond Distributed Transactions: an Apostate’s Opinion,” in *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, Asilomar, CA, USA, 2007.
- [8] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Philadelphia, PA, USA, 2014, pp. 305–319.
- [9] X/Open Company Ltd., “Distributed Transaction Processing: The XA Specification,” X/Open Document XO/CAE/91/300, 1991.
- [10] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [11] J. Gray and L. Lamport, “Consensus on Transaction Commit,” *ACM Transactions on Database Systems*, vol. 31, no. 1, pp. 133–160, Mar. 2006.
- [12] M. Abdallah and P. Pucheral, “A Performance Comparison of Distributed Commit Protocols,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 2, pp. 195–213, Feb. 2004.
- [13] J. Brzeziński and D. Wawrzyniak, “An Analysis of Distributed Commit Protocols,” in *Proceedings of the International Conference on Parallel Processing and Applied Mathematics (PPAM)*, Poznań, Poland, 2005, pp. 957–964.
- [14] W. Vogels, “Eventually Consistent,” *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, Jan. 2009.
- [15] M. Stonebraker, “The Case for Shared Nothing,” in *Proceedings of the International Workshop on Database Machines*, Monterey, CA, USA, 1985, pp. 1–22.
- [16] L. Lamport, “Paxos Made Simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, Dec. 2001.
- [17] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation: Write-Ahead Logging (WAL).” 2023.
- [18] C. Mohan, B. Lindsay, and R. Obermarck, “Transaction Management in the R* Distributed Database Management System,” *ACM Transactions on Database Systems*, vol. 11, no. 4, pp. 378–396, Dec. 1986.
- [19] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. New York, NY, USA: Springer, 2020.

IX. Apéndice

A. Código Fuente

Repositorio del Proyecto

B. Reproducibilidad

Todos los experimentos son reproducibles mediante:

- 1) `cd docker && docker compose up -d --build`
- 2) `cd experiments && sh run_experiments.sh`
- 3) `python analyze_results.py results graphs`

El código fuente completo está disponible en el repositorio del laboratorio.